An overall description of the program and how it demonstrates each of the concepts listed in the "Final Project" section.

We plan to create a Java application that uses machine learning to predict stock returns and test trading strategies on historical data. Users will import stock data from CSV files, choose a strategy, and simulate investing a starting amount of virtual money. The program will display trades and a graph comparing the strategy's portfolio value with buy-and-hold.

We will use ridge regression to predict returns from recent price changes, moving averages, and volatility. We plan to implement the model using gradient descent. The strategy will rank stocks by predicted returns and select the highest-ranked stocks with positive predictions. Historical volatility will help determine how much money to allocate to each stock. We will also include a moving-average strategy from comparison.

We will train the model on earlier data and evaluate it on later data. The simulation will include transaction costs and only use information available before each trade. We will compare total returns and maximum drawdown, the largest percentage decline from a previous portfolio peak. The project's success will depend on building and evaluating the program correctly, even if the model does not outperform buy and hold.

For inheritance and abstract classes, we will create an abstract TradingStrategy class extended by MachineLearningStrategy, MovingAverageStrategy, and BuyAndHoldStrategy. Each subclass will override its stock-selection method. A PredictionModel interface will define training and prediction methods implemented by RidgeRegressionModel. The backtesting engine will demonstrate method overloading through **run** methods that accept either default or custom settings.

We will use ArrayList objects for historical prices and completed trades, and hashmaps to connect stock symbols with price histories and portfolio holdings. A manually implemented linked queue will hold pending orders until execution. This queue will use a generic type, LinkedQueue<T>, contain a private inner Node class, and provide an iterator for inspecting orders without removing them. We will also manually implement a binary max-heap to rank stocks by predicted return.

A Swing GUI will provide file selection, strategy settings, a results table, and a portfolio chart. Lambda expressions will connect buttons to actions such as importing data and running simulations, demonstrating event-driven programming. Our JUnit suit will test the data structures, mathematical calculations, model, and portfolio accounting.

We will keep fields private and provide methods that control how data is accessed or changed. The GUI will be separate from the mathematical calculations so we can test those calculations independently. We will handle file-reading exceptions and invalid CSV values with error messages. Our tests will check queue ordering, heap ranking, indicator calculations, and model behavior on synthetic data. We will also test transaction costs and verify that future prices cannot affect earlier trading decisions.

Your team's communication and collaboration plan. How will you talk to each other? How will you share code? When and how will your group meet up (in person or virtually)?

We are both familiar with GitHub and have each other's personal contacts, so we can coordinate with each other through those channels and share/version control code. We plan to meet up virtually every two or three days to share progress and discuss what step we should tackle next.

A tentative week-by-week timeline. (This will almost certainly change as the project progresses.)

Week	Planned Progress
November 9	Finalize the design, setup GitHub and JUnit, obtain historical data, and implement file I/O for CSVs
November 16	Build and test the manual queue and heap. Implement portfolio accounting and a working buy-and-hold simulation
November 23	Implement mathematical features, ridge regression, and other trading strategies. Establish training and testing periods

November 30	Connect the program to the GUI. Add charts and results tables, then test transaction costs and error handling
December 7	Compare strategy results, fix remaining problems, finish documentation, and prepare the demonstration

How the project will contribute to each team member's goals for this course. (Think back to the class goals you set for yourself in ICE01.)

Abhi's goals: My personal goals for the term surround comfort with OOP, so through this project's concepts of inheritance, abstract classes, and general OOP principles, I hope to get closer to proficiency and comfort of OOP.

Richy's goals: I want more practice building Java projects that go beyond a single Java file for an assignment. I've worked with math modeling and machine learning before, but this project will give me experience turning these models into a program that others can see and use rather than being stuck on my computer.